

Python Dependency Management & CLI Flags: Lab Manual

Institute Handout – Beginner Programming Series

July 9, 2026

Lab Objectives

By the end of this lab session, students will be able to:

1. Isolate software dependencies using Python's native `venv` engine.
2. Install, manage, and audit external packages cleanly via `pip`.
3. Package environment requirements profiles using a `requirements.txt` manifest.
4. Execute and manipulate interpreter behaviors using common CLI flags (`-m`, `-c`, `-i`).

Safety Warning for Beginners

Isolation Guardrail: Always ensure that your local environment prefix prompt indicator (`.venv`) is fully active on the left side of your terminal path strings before firing off any `pip install` actions. This safely avoids polluting or breaking your operating system's global system-wide Python installation profile.

1 Task 1: Creating and Activating Environment Sandboxes

Before importing libraries like Django, we must build an isolated sandbox directory.

1. Open your target shell (Command Prompt or PowerShell) and step onto your Desktop:

```
cd Desktop
```

2. Create a dedicated project sandbox folder and navigate immediately inside it:

```
mkdir environment_lab  
cd environment_lab
```

3. Compile the new virtual environment binary structural folders using the command below:

```
python -m venv .venv
```

4. **Activation Strategy:** Run the routing trigger file matching your active shell environment setup:

Command Prompt (CMD):

```
.venv\Scripts\activate.bat
```

PowerShell Script (PS):

```
.venv\Scripts\Activate.ps1
```

2 Task 2: Dependency Manipulation Natively via pip

With isolation established, we can download and configure historical version variations of web frameworks.

1. Interrogate the base profile setup directory to see what libraries exist by default:

```
pip list
```

2. Safely download and compile an explicit older version release of Django:

```
pip install django==4.2.5
```

3. Run your environment listing tool again to verify how sub-dependencies automatically mapped along with your installation:

```
pip list
```

3 Task 3: Generating and Restoring Ecosystem Blueprints

To keep project code lightweight, we never share or commit the massive structural `.venv` directory. Instead, we export a text-based recipe manifest profile.

1. Freeze and capture your unique local package ecosystem footprint inside a recipe manifest:

```
pip freeze > requirements.txt
```

2. **Simulate Environment Loss:** Unconditionally purge Django to test your manual tracking limits:

```
pip uninstall django -y
```

3. **Restore Environment via Manifest:** Instruct your engine to automatically re-download and align the software from your blueprint recipe:

```
pip install -r requirements.txt
```

4 Task 4: Harnessing Command Line Execution Flags

Python features highly functional runtime execution configurations passed right during initial shell calling scopes.

4.1 Part A: Executing Libraries as Utility Scripts (-m)

1. Explicitly route traffic to pip bound to the active terminal shell instance cleanly:

```
python -m pip list
```

2. Launch a lightweight local network file browser server out of your current directory:

```
python -m http.server 8000
```

3. Open up your browser and visit `http://localhost:8000` to see your local files rendered directly on the web. Press `Ctrl + C` inside your shell to kill the process.

4.2 Part B: Processing Inline Script Code On-The-Fly (-c)

1. Execute simple logical lines without creating messy, throwaway text files:

```
python -c "print('Direct terminal code execution running successfully.')" 
```

2. Use inline processing to programmatically inspect internal system environment runtime paths:

```
python -c "import sys; print(sys.path)" 
```

4.3 Part C: Interactive Debugging Post-Execution (-i)

1. Build a simple mathematical script document named `math_test.py`:
CMD / PowerShell Shared Shortcut Command:

```
echo x = 105 > math_test.py 
```

2. Run the file standardly using the standard interpreter loop:

```
python math_test.py 
```

Notice that the execution finishes instantly and silently terminates back to the baseline path prompt.

3. Now, override the termination step and force the environment to remain open for analysis by appending the inspection flag:

```
python -i math_test.py 
```

4. Notice your command terminal switches to a three-chevron prompt (`>>>`). Evaluate your running variable status live:

```
print(x) 
```

5. Exit the interactive terminal completely to finish your lab profile:

```
exit() 
```

Lab Exit & Cleanup Sequence

Instruct the shell session to safely detach and power down your sandbox session container by executing:

```
deactivate 
```

Verify the `(.venv)` tag has cleanly dropped from your active terminal dashboard line.